

Amagi Framework v1.3: Architecture as Compliance — A Structural Guarantee for High-Assurance AI Systems

Author: Alexey Mikhailovich Burlai

[ORCID: 0009-0001-4679-5967](https://orcid.org/0009-0001-4679-5967)

Affiliation: Independent Researcher, Sochi, Russia

Date: 2025-11-24

Preprint DOI: [10.5281/zenodo.17696607](https://doi.org/10.5281/zenodo.17696607)

License: [CC BY-NC 4.0](https://creativecommons.org/licenses/by-nc/4.0/)

Commercial Use: Requires a valid license under the Strict Architectural Principle License (S APL), governed by the laws of England and Wales

Contact: lexintrud@gmail.com

Abstract

Current compliance methodologies for high-risk **Artificial Intelligence (AI)** systems, as mandated by frameworks like the **EU AI Act (Annex III)**, rely predominantly on procedural controls and software-based policies. We argue that procedural control is insufficient for **high-assurance systems**, where the cost of failure is unacceptable. This paper presents the **Amagi Framework**, a reference architecture that implements the principle of **Separation of Control** through a **new class of specialized, high-assurance hardware components**. By introducing the **TTP Triad (Trigger–Programmable Delay–Localized Activation)**, we physically isolate "untrusted" cognitive logic from a "trusted" safety control plane. We formalize a threat model that treats the AI model as a hostile insider, define the **Security Council Core (SCC)** as a **Trusted Computing Base (TCB)**, and demonstrate how the architecture provides structural guarantees of compliance — specifically **Non-bypassability** and **Complete Mediation** — regardless of the stochastic nature of the underlying model.

Keywords: High-Assurance AI, Separation of Control, TTP Formalism, Hardware Root of Trust, EU AI Act, CHERI, seL4, Architecture as Compliance

1. Introduction and Context

The safety of AI systems is traditionally addressed at the MLOps layer (e.g., weight monitoring, data filtering). However, in **life-critical** and **mission-critical** domains — such as medical implants, avionics, and infrastructure control — software-based guarantees are inherently fragile.

The complexity of modern software stacks, coupled with the risk of privilege escalation, severely limits the trust we can place in purely procedural controls.

The **Amagi Framework** does not propose minor incremental changes, but introduces a **new class of specialized hardware components** designed to deliver *fundamental safety guarantees by construction*. Rather than relying on software fixes, these components embed safety at the architectural level, enabling AI systems to operate in high-risk environments with structurally enforced trust.

1.1 Related Work

Amagi builds on decades of research in secure systems architecture:

- **Lampson’s Protection Model (1971):** We realize the access-matrix concept in hardware via the **Memory Access Graph (MAG)**.
- **Reference Monitor Concept (Anderson, 1972):** The **SCC** acts as a hardware-level reference monitor, enforcing mediation of all privileged operations.
- **Separation Kernels & seL4 (Rushby, 1981):** We adapt mathematically proven isolation concepts from OS kernel design to chip-scale isolation (**Network-on-Chip**, or NoC).
- **Capability-based Addressing (CHERI):** Similar to CHERI, Amagi restricts access using unforgeable capabilities, but augments this with **temporal context** via the **TTP principle**.

1.2 Scope and Non-Goals

To avoid misunderstanding, we delineate what Amagi is **not** and what it **does**:

Non-Goals:

1. **Data Quality:** Amagi does not fix bias in training datasets.
2. **Algorithmic Correctness:** It does not guarantee that the AI makes optimal or morally “correct” decisions.
3. **Global Interpretability:** It does not solve the “black box” problem in the general sense.

Primary Goal: Amagi guarantees **Containment**. Even if an AI model is faulty, hallucinating, or compromised, it cannot perform unsafe or unapproved operations that violate physical safety invariants.

Deployment Intent: Amagi is *not* intended to replace general-purpose CPUs or memory in consumer devices. Rather, it is designed to be integrated as a **dedicated safety co-processor** or as part of a **heterogeneous SoC**, working side by side with conventional computing units for high-risk applications.

2. Formalizing the TTP Principle

At the heart of Amagi is the **TTP Triad** — **Trigger, Programmable Delay, Localized Activation**. We formalize this via a predicate that must be satisfied for any **critical operation** (Op) :

$$Execute(Op) \iff Triggered(Op) \wedge (\Delta_{ctx} \geq \Delta_{min}) \wedge Policy(Ctx, Op)$$

Where:

- $\text{\$Op \in Operations\$}$: A set of privileged instructions (e.g., actuator writes, access to cryptographic keys).
- $\text{\$Ctx \in Context\$}$: The verifiable system state (e.g., timestamp, geolocation, biometric signature).
- $\text{\$}\Delta_{\text{ctx}} \in \mathbb{R}^+\text{\$}$: The delay introduced by the hardware to validate the context.
- $\text{\$}\Delta_{\text{min}}\text{\$}$: A hardware-enforced minimum delay that must elapse before activation (a hard constraint).
- $\text{\$Triggered(Op)\$}$: A boolean indicating whether a valid physical trigger signal is present.
- $\text{\$Policy(Ctx, Op)\$}$: A function, defined by the **Memory Access Graph (MAG)**, that verifies whether the operation is permitted in the current context.

If the predicate evaluates to **False**, the operation is blocked at the hardware gate, enforcing a **fail-safe** posture.

3. Hardware Implementation and Security Properties

3.1 High-Level Architecture

The architecture implements **Separation of Control** by physically isolating the high-risk cognitive logic from the control path.

Figure 1: Topology of Amagi system. The **Cognitive Cluster** (untrusted) is physically isolated from actuators and external I/O. All privileged control signals must go through the **Security Council Core (SCC)**, which references the **Memory Access Graph (MAG)** stored in immutable memory (e.g., OTP).

3.2 Formal Security Invariants

Amagi enforces three core invariants:

1. Complete Mediation: All memory access requests $\text{\$(M)\$}$ from any subject $\text{\$(S)\$}$ must traverse a Memory Policy Enforcer (MPE). There is no bypass path:

$$\text{\$}\text{Path}(S, M) \cap \text{MPE} \neq \emptyset\text{\$}$$

2. **Non-bypassability**: The **Cognitive Cluster** lacks any direct physical connection to I/O or actuators. Messages or commands can only be routed via the **SCC**.
3. No Privilege Escalation: The MAG is immutable once programmed (e.g., in OTP), ensuring that runtime cannot alter critical security policy:

$$\text{\$}\text{MAG}_{\{\text{runtime}\}} \equiv \text{MAG}_{\{\text{boot}\}}\text{\$}$$

3.3 Security Council Core (SCC)

The **SCC** acts as the **Trusted Computing Base (TCB)** with the following properties:

- **Static Linking:** Software running on the SCC is statically linked; no dynamic code loading is permitted.
- **Memory Safety:** Implemented in memory-safe languages (e.g., SPARK Ada, Rust) to reduce risks of memory corruption.
- **Hard Real-Time Guarantees:** Execution is deterministic with a guaranteed Worst-Case Execution Time (WCET).
- **Air-Gapped Logic:** It does not directly interface with untrusted external networks — all communication is mediated and controlled.

3.4 Memory Access Graph (MAG)

The **MAG** defines fine-grained, typed access rules. Example configuration (in YAML):

```
YAML
- Source: Cluster_AI_Inference      # Type: ClusterID (0x0A)
  Target: Memory_Region_Weights    # Type: RegionID (0xFF00-0xFFFF)
  Action: READ_ONLY                # Enum: [READ, WRITE, DENY]
  Condition: TTP_Verified          # Bool: Hardware Flag Check

- Source: Cluster_AI_Inference
  Target: Memory_Region_RootKeys
  Action: DENY                    # Result: Hardware Interrupt (Trap)
```

Each rule is stored in write-once memory and is enforced by the SCC/MPE on every access attempt.

4. Threat Model

We model the **Cognitive Module (AI)** as a **Hostile Insider** capable of arbitrary (or faulty) behavior. Key threat vectors and mitigations:

Attack Vector	Mitigation Strategy
Insider Threat (Compromised AI)	Isolation: AI cannot directly influence actuators; SCC mediates all sensitive operations via MAG.
Privilege Escalation	Immutable Policy: MAG is fixed and cannot be modified at runtime.
Side-Channel Attacks	Default Inertia: The system remains in low-power “sleep” mode until a valid physical trigger is received, preventing attackers from easily inducing exploitable side-effects.
Supply Chain Injection	Audit Logging: All policy violations, access attempts, and rejections are recorded in a cryptographically protected, append-only log (e.g., with post-quantum signatures).

5. Architectural Integration and Case Study

5.1 System-Level Integration

The Amagi architecture is designed to integrate as a co-processor in a **heterogeneous SoC**, ensuring that safety guarantees are provided without disrupting the main computational workflow.

Figure 2: System-level integration of Amagi in a heterogeneous SoC. This demonstrates that **Amagi does not replace** existing compute systems; it complements them by offering high-assurance hardware for critical control paths.

5.2 Case Study: Amagi-H (Medical Pacemaker)

Scenario: An implantable, AI-enabled pacemaker.

1. **Event:** The AI module detects arrhythmia and proposes a shock ($Op_{\{shock\}}$).
2. **TTP Verification:**
 - *Trigger:* A biosensor produces an analog signal confirming arrhythmia.
 - *Delay:* Hardware enforces a 10 ms delay ($\Delta_{\min}$) to perform a deterministic safety algorithm check.
 - *Policy Check:* The safety algorithm verifies that heart rate $\leq 40 \text{ bpm}$ (context $Ctx_{\{valid\}}$). MAG's policy permits the operation ($Policy(Ctx, Op_{\{shock\}}) = True$).
3. **Outcome:** Shock is authorized by the SCC.
 - *Counter-factual:* If the AI had hallucinated arrhythmia when the heart rate was normal, the SCC would block the command because the policy check would return **False**.

5.3 Performance Overhead

The TTP mechanism introduces a hardware delay, but the SCC runs in parallel on a dedicated **NoC**, making this overhead **deterministic**. For typical control loop latencies (10–100 ms), the policy check ($\sim 10 \mu s$) is minimal and does not violate real-time constraints. Amagi's design **optimizes for predictability and trust**, which is the necessary and justified specialization for high-assurance domains.

6. Security Equivalence Criteria

To facilitate certification and derivative design, we define equivalence criteria:

Criterion	Requirement
Isolation	Proven physical or formal separation between the control plane (SCC) and the cognitive plane.

Criterion	Requirement
Determinism	The SCC (TCB) must guarantee a bounded Worst-Case Execution Time (WCET) .
Immutability	Security policies (MAG) must reside in write-once or OTP memory.
Auditability	All security-relevant events (e.g., denied accesses) must be logged with post-quantum cryptographic signatures.

7. Conclusion

The **Amagi Framework** demonstrates that **structural compliance and safety** can be built *into hardware*, not just tacked on in software. By introducing a **new paradigm** — specialized, high-assurance hardware implementing **Trigger–Programmable Delay–Localized Activation** — we shift from the fragile model of "**trust through testing**" to a more robust model of "**trust through construction**." Rather than replacing all computing hardware, Amagi establishes a **higher-assurance class** of hardware tailored for domains where failure is not an option.

References

1. Burlai, A. M. (2025). *Amagi Framework v1.0*. Zenodo. DOI: [10.5281/zenodo.17679248](https://doi.org/10.5281/zenodo.17679248)
 2. Burlai, A. M. (2025). *Amagi-TTP NIPU v1.0*. Zenodo. DOI: [10.5281/zenodo.17682668](https://doi.org/10.5281/zenodo.17682668)
 3. Burlai, A. M. (2025). *Amagi-MEM v1.0*. Zenodo. DOI: [10.5281/zenodo.17679327](https://doi.org/10.5281/zenodo.17679327)
 4. Lampson, B. W. (1971). "Protection." *5th Princeton Conference on Information Sciences and Systems*.
 5. Anderson, J. P. (1972). *Computer Security Technology Planning Study*, USAF Electronic Systems Division.
 6. Rushby, J. (1981). "Design and Verification of Secure Systems." *ACM SOSP*.
 7. Watson, R. N., et al. (2015). "CHERI: A Hybrid Capability-System Architecture." Cambridge Technical Report.
 8. Klein, G., et al. (2009). "seL4: Formal Verification of an OS Kernel." *ACM SOSP*.
 9. ISO. (2018). *ISO 26262: Road Vehicles — Functional Safety*.
 10. European Commission. (2024). *Regulation on Artificial Intelligence ("AI Act")*.
-

© 2025 Alexey M. Burlai. All rights reserved.